

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный университет им.
Н.И. Лобачевского

Институт информационных технологий, математики и механики

**УЧЕБНЫЙ ПРОЕКТ ПО ДИСЦИПЛИНЕ
«ТЕХНОЛОГИИ БАЗ ДАННЫХ»**

на тему:

«Сдача в аренду торговых площадей»

Выполнил:

студент группы 3824Б1ПР1
Черкасова Полина Денисовна

Проверил:

доцент кафедры МОСТ, к.ф.-м.н.
Шапошников Д.Е.

Нижний Новгород
2026

Постановка задачи проекта

Целью данной работы является разработка реляционной базы данных для информационной системы торгового центра, сдающего в аренду торговые площади.

В ходе работы необходимо:

- спроектировать структуру БД на основе описанной предметной области;
- реализовать таблицы, ограничения целостности и связи;
- подготовить аналитические запросы с использованием агрегации, соединений, подзапросов и скалярных функций;
- реализовать триггер, хранимую процедуру с курсором, процедуру каскадного удаления без CASCADE;
- добавить гибкую схему EAV для необязательных атрибутов;
- построить аналитический набор данных с расчётом рейтинга и отрыва от следующего объекта.

Описание предметной области (исходная версия)

Торговый центр сдаёт в аренду коммерсантам свои площади.

Для каждой **торговой точки** фиксируется:

- этаж
- площадь
- наличие кондиционера
- стоимость аренды в день

По каждому **клиенту** хранится:

- название
- реквизит
- адрес
- телефон
- контактное лиц

При заключении договора в таблице «**Аренда**» фиксируется:

- торговая точка
- клиент
- дата начала и окончания аренды

Первичные ключи генерируются автоматически с использованием последовательностей.

Реализация исходной модели БД

На основе спроектированной схемы была создана структура базы данных в СУБД PostgreSQL. Ниже представлены:

- SQL-скрипты создания таблиц с ограничениями;
- скриншоты из pgAdmin, подтверждающие структуру;
- примеры запросов с результатами выполнения.

```
CREATE TABLE public.stores (
  store_id SERIAL NOT NULL,
  floor integer NOT NULL,
  area numeric(8,2) NOT NULL,
  has_ac boolean,
  daily_cost numeric(10,2) NOT NULL,
  CONSTRAINT stores_pkey PRIMARY KEY (store_id),
  CONSTRAINT floor_check CHECK ((floor > 0)),
  CONSTRAINT area_check CHECK ((area > (0)::numeric)),
  CONSTRAINT cost_check CHECK ((daily_cost > (0)::numeric))
);
```

```
INSERT INTO public.stores (floor, area, has_ac, daily_cost) VALUES
(1, 45.50, true, 1500.00),
(2, 32.00, false, 800.00),
(1, 78.30, true, 2500.00),
(3, 28.50, true, 900.00),
(2, 120.00, false, 3000.00),
(1, 15.00, false, 500.00),
(2, 95.00, true, 2800.00),
(3, 42.00, false, 1100.00),
(1, 60.00, true, 2000.00),
(2, 35.00, true, 1000.00),
(1, 200.00, true, 5000.00),
(3, 18.50, false, 600.00);
```

```
arenda=# select * from stores;
 store_id | floor | area  | has_ac | daily_cost
-----+-----+-----+-----+-----
        1 |     1 | 45.50 | t      |    1500.00
        2 |     2 | 32.00 | f      |     800.00
        3 |     1 | 78.30 | t      |    2500.00
        4 |     3 | 28.50 | t      |     900.00
        5 |     2 | 120.00 | f      |    3000.00
        6 |     1 | 15.00 | f      |     500.00
        7 |     2 | 95.00 | t      |    2800.00
        8 |     3 | 42.00 | f      |    1100.00
        9 |     1 | 60.00 | t      |    2000.00
       10 |     2 | 35.00 | t      |    1000.00
       11 |     1 | 200.00 | t      |    5000.00
       12 |     3 | 18.50 | f      |     600.00
(12 rows)
```

```
CREATE TABLE public.clients (
  client_id SERIAL NOT NULL,
  name character varying(100) NOT NULL,
  requisites text,
  address text,
  phone character varying(20),
  contact_person character varying(100),
  CONSTRAINT clients_pkey PRIMARY KEY (client_id)
);
```

```
INSERT INTO public.clients (name, requisites, address, phone, contact_person) VALUES
('Nike', '123456789', 'Moscow', '79991112233', 'Ivan Petrov'),
('Adidas', '987654321', 'Saint Petersburg', '79994445566', 'Anna Smirnova'),
('Zara', '456123789', 'Kazan', '79997778899', 'Olga Ivanova'),
('MVideo', '987000123', 'Ufa', '79261112233', 'Vasilyev Oleg'),
('Puma', '111222333', 'Novosibirsk', '79001112233', 'Alexey Kozlov'),
('Reebok', '444555666', 'Ekaterinburg', '79004445566', 'Elena Morozova'),
('H&M', '777888999', 'Samara', '79007778899', 'Dmitry Volkov'),
('Ozon', '333888555', 'Voronezh', '79263334455', 'Gromov Sergey'),
('Uniqlo', '000111222', 'Rostov-on-Don', '79001110022', 'Maria Sokolova'),
('Decathlon', '333444555', 'Krasnodar', '79003334455', 'Pavel Lebedev');
('MyStore', '5551237890', 'Nizhny Novgorod', '+79998887766', 'Petr Smirnov'),
('ElectroWorld', '4445556667', 'Moscow', '+79995554433', 'Elena Volkova'),
('Fashion', '7778889990', 'Kazan', '+79991110022', 'Igor Morozov');
```

```
arenda=# select * from clients;
```

client_id	name	requisites	address	phone	contact_person
1	Nike	123456789	Moscow	+79991112233	Ivan Petrov
2	Adidas	987654321	Saint Petersburg	+79994445566	Anna Smirnova
3	Zara	456123789	Kazan	+79997778899	Olga Ivanova
4	MVideo	987000123	Ufa	+79261112233	Vasilyev Oleg
5	Puma	111222333	Novosibirsk	+79001112233	Alexey Kozlov
6	Reebok	444555666	Ekaterinburg	+79004445566	Elena Morozova
7	H&M	777888999	Samara	+79007778899	Dmitry Volkov
8	Ozon	333888555	Voronezh	+79263334455	Gromov Sergey
9	Uniqlo	000111222	Rostov-on-Don	+79001110022	Maria Sokolova
10	Decathlon	333444555	Krasnodar	+79003334455	Pavel Lebedev
11	MyStore	5551237890	Nizhny Novgorod	+79998887766	Petr Smirnov
12	ElectroWorld	4445556667	Moscow	+79995554433	Elena Volkova
13	Fashion	7778889990	Kazan	+79991110022	Igor Morozov

(13 rows)

```
CREATE TABLE public.rentals (
  rental_id SERIAL NOT NULL,
  store_id SERIAL NOT NULL,
  client_id SERIAL NOT NULL,
  start_date date NOT NULL,
  end_date date NOT NULL,
  CONSTRAINT rentals_pkey PRIMARY KEY (rental_id),
  CONSTRAINT rentals_client_id_fkey FOREIGN KEY (client_id) REFERENCES public.clients(client_id) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT rentals_store_id_fkey FOREIGN KEY (store_id) REFERENCES public.stores(store_id) ON UPDATE CASCADE ON DELETE RESTRICT
);
```

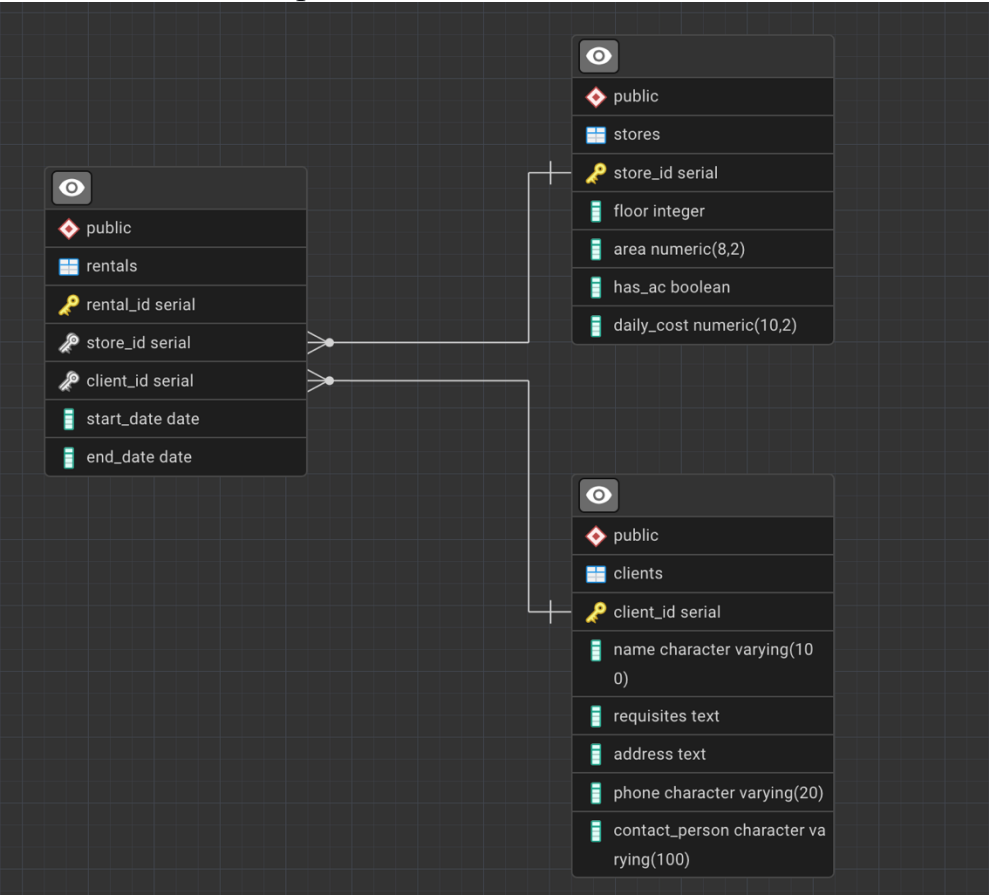
```
INSERT INTO public.rentals (store_id, client_id, start_date, end_date) VALUES
(1, 1, '2026-01-15', '2026-12-31'),
(2, 2, '2026-02-01', '2026-08-31'),
(3, 3, '2026-03-10', '2026-09-10'),
(4, 4, '2026-01-20', '2026-07-20'),
(5, 5, '2026-04-01', '2026-10-31'),
(6, 6, '2026-02-15', '2026-08-15'),
(7, 7, '2026-05-01', '2026-11-30'),
(8, 8, '2026-03-20', '2026-09-20'),
(9, 9, '2026-06-01', '2026-12-31'),
(10, 10, '2026-04-15', '2026-10-15'),
(11, 1, '2026-07-01', '2026-12-31'),
(12, 3, '2026-05-10', '2026-11-10'),
(4, 5, '2026-08-01', '2027-01-31'),
(6, 7, '2026-06-15', '2026-12-15'),
(8, 2, '2026-09-01', '2027-02-28');
```

arenda=# select * from rentals;

rental_id	store_id	client_id	start_date	end_date
1	1	1	2026-01-15	2026-12-31
2	2	2	2026-02-01	2026-08-31
3	3	3	2026-03-10	2026-09-10
4	4	4	2026-01-20	2026-07-20
5	5	5	2026-04-01	2026-10-31
6	6	6	2026-02-15	2026-08-15
7	7	7	2026-05-01	2026-11-30
8	8	8	2026-03-20	2026-09-20
9	9	9	2026-06-01	2026-12-31
10	10	10	2026-04-15	2026-10-15
11	11	1	2026-07-01	2026-12-31
12	12	3	2026-05-10	2026-11-10
13	4	5	2026-08-01	2027-01-31
14	6	7	2026-06-15	2026-12-15
15	8	2	2026-09-01	2027-02-28

(15 rows)

Все зависимости представлены ниже



Операторы языка SQL для вывода агрегатных данных

1. Сколько каждый клиент платит в день за аренду всех своих точек?
(соединение таблиц)

```
SELECT client_id, SUM(daily_cost) AS total_sum
FROM stores JOIN rentals ON stores.store_id = rentals.store_id
GROUP BY client_id ORDER BY client_id;
```

client_id	total_sum
1	6500.00
2	1900.00
3	3100.00
4	900.00
5	3900.00
6	500.00
7	3300.00
8	1100.00
9	2000.00
10	1000.00

(10 rows)

2. Сколько осталось месяцев до конца аренды от сегодняшней даты у каждой торговой точки? (скалярные функции работы с датой)

```
SELECT rental_id, AGE(end_date, CURRENT_DATE) AS days_remaining
FROM rentals WHERE end_date >= CURRENT_DATE ORDER BY rental_id;
```

rental_id	days_remaining
1	7 mons
2	3 mons
3	3 mons 10 days
4	1 mon 20 days
5	5 mons
6	2 mons 15 days
7	5 mons 30 days
8	3 mons 20 days
9	7 mons
10	4 mons 15 days
11	7 mons
12	5 mons 10 days
13	8 mons
14	6 mons 15 days
15	8 mons 28 days

(15 rows)

3. Вывести клиентов, у которых телефона заканчивается на «5».
(скалярные функции с текстовым поиском)

```
SELECT client_id, name, phone FROM clients WHERE phone LIKE '%5';
```

client_id	name	phone
8	Ozon	+79263334455
10	Decathlon	+79003334455

(2 rows)

4. Запрос на клиентов, у которых нет арендованного магазина (поиск родительских записей, у которых нет дочерних)

```
SELECT name, store_id FROM clients  
LEFT JOIN rentals ON clients.client_id = rentals.client_id WHERE store_id IS NULL;
```

name	store_id
MyStore	
ElectroWorld	
Fashion	

(3 rows)

Развитие постановки задачи

В процессе развития информационной системы были выявлены новые требования. В первоначальной версии предполагалось, что одна запись в таблице аренды соответствует одной торговой точке. В дальнейшем возникла необходимость учитывать случаи, когда один клиент арендует несколько торговых точек одновременно в рамках одного договора. Кроме того, нужно вести учёт ежемесячных платежей по договорам аренды и выполнять аналитическую обработку данных.

Для реализации новых требований структура базы данных была расширена:

- Была изменена таблицы rentals: поле store_id удалено

```
ALTER TABLE rentals DROP COLUMN store_id;
```

- Для реализации связи «один договор – несколько торговых точек» была введена промежуточная таблица rental_stores

```
CREATE TABLE rental_stores (  
    rental_store_id SERIAL PRIMARY KEY,  
    rental_id INTEGER NOT NULL,  
    store_id INTEGER NOT NULL,
```

```

CONSTRAINT fk_rental FOREIGN KEY (rental_id) REFERENCES rentals(rental_id),
CONSTRAINT fk_store FOREIGN KEY (store_id) REFERENCES stores(store_id)
);

```

```

INSERT INTO public.rental_stores (rental_id, store_id) VALUES
(1, 1), (1, 11), (2, 2), (3, 3), (3, 12), (4, 4), (5, 5), (5, 10),
(6, 6), (7, 7), (8, 8), (9, 9), (10, 10), (11, 11), (11, 1), (12, 12),
(12, 3), (13, 4), (13, 9), (14, 6), (14, 7), (15, 8), (15, 2);

```

```

arenda=# select * from rental_stores;
 rental_store_id | rental_id | store_id
-----
          1 |         1 |         1
          2 |         1 |        11
          3 |         2 |         2
          4 |         3 |         3
          5 |         3 |        12
          6 |         4 |         4
          7 |         5 |         5
          8 |         5 |        10
          9 |         6 |         6
         10 |         7 |         7
         11 |         8 |         8
         12 |         9 |         9
         13 |        10 |        10
         14 |        11 |        11
         15 |        11 |          1
         16 |        12 |        12
         17 |        12 |          3
         18 |        13 |          4
         19 |        13 |          9
         20 |        14 |          6
         21 |        14 |          7
         22 |        15 |          8
         23 |        15 |          2
(23 rows)

```

- Для учёта ежемесячных платежей по договорам аренды была разработана таблица payments. Таблица хранит информацию о дате платежа, сумме платежа и его текущем статусе.

```

CREATE TABLE payments (
    payment_id SERIAL PRIMARY KEY,
    rental_id INTEGER NOT NULL,
    payment_date DATE NOT NULL,
    amount NUMERIC(10,2) NOT NULL CHECK (amount > 0),
    payment_status VARCHAR(20) NOT NULL,
    CONSTRAINT fk_payment_rental FOREIGN KEY (rental_id) REFERENCES rentals(rental_id)
);

```

```

INSERT INTO public.payments (rental_id, payment_date, amount, payment_status) VALUES
(1, '2026-02-01', 45000.00, 'Paid'), (1, '2026-03-01', 45000.00, 'Pending'),
(1, '2026-04-01', 45000.00, 'Paid'), (2, '2026-03-01', 24000.00, 'Paid'),
(2, '2026-04-01', 24000.00, 'Paid'), (3, '2026-04-01', 75000.00, 'Paid'),
(3, '2026-05-01', 75000.00, 'Overdue'), (4, '2026-02-20', 27000.00, 'Paid'),
(5, '2026-05-01', 90000.00, 'Paid'), (5, '2026-06-01', 90000.00, 'Pending'),
(6, '2026-03-15', 15000.00, 'Overdue'), (7, '2026-06-01', 84000.00, 'Paid'),
(8, '2026-04-20', 33000.00, 'Paid'), (9, '2026-07-01', 60000.00, 'Paid'),
(10, '2026-05-15', 30000.00, 'Paid'), (11, '2026-08-01', 150000.00, 'Paid'),
(12, '2026-06-10', 18000.00, 'Paid'), (13, '2026-09-01', 54000.00, 'Paid'),
(14, '2026-07-15', 30000.00, 'Overdue'), (15, '2026-10-01', 22000.00, 'Pending');

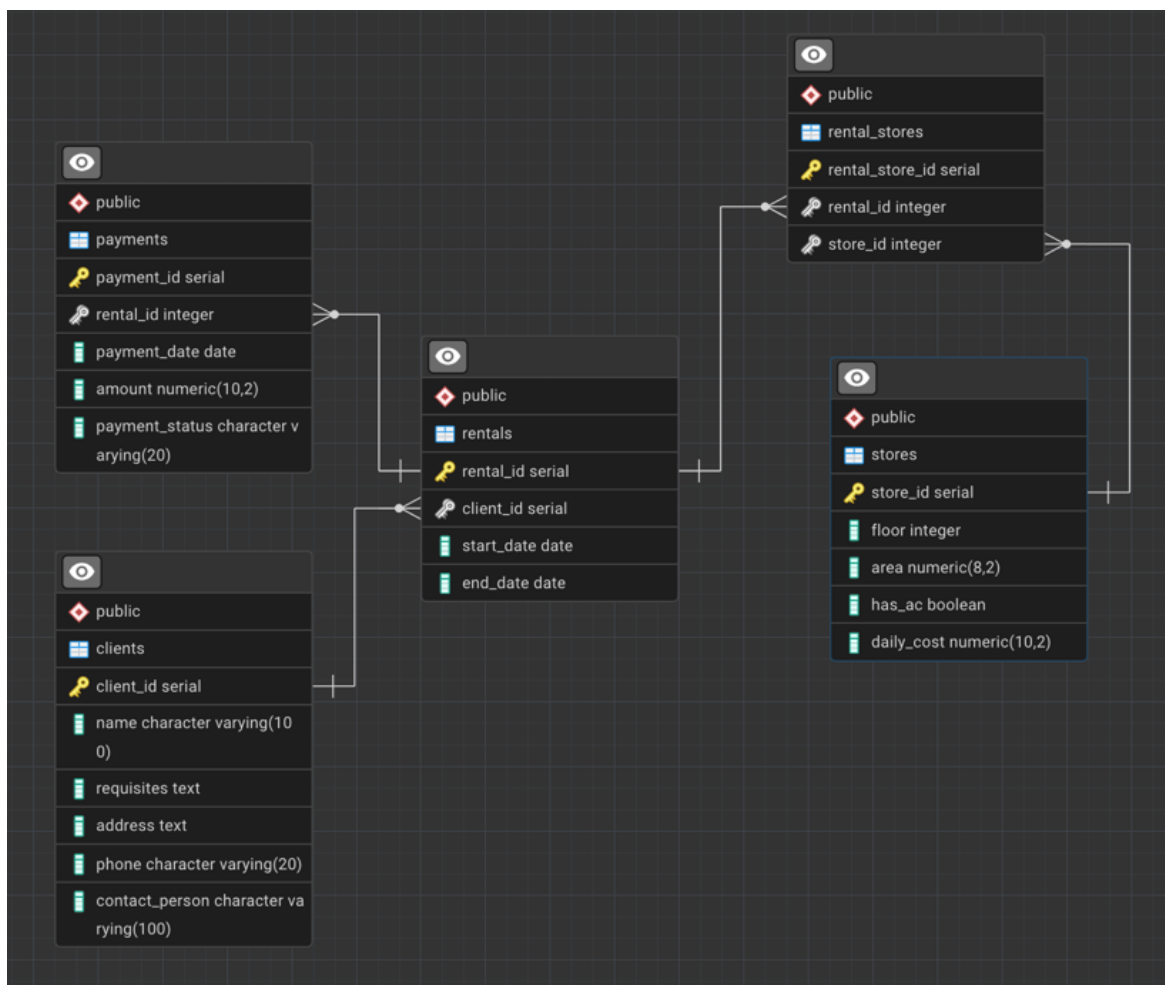
```

```
arenda=# select * from payments;
```

payment_id	rental_id	payment_date	amount	payment_status
1	1	2026-02-01	45000.00	Paid
2	1	2026-03-01	45000.00	Pending
3	1	2026-04-01	45000.00	Paid
4	2	2026-03-01	24000.00	Paid
5	2	2026-04-01	24000.00	Paid
6	3	2026-04-01	75000.00	Paid
7	3	2026-05-01	75000.00	Overdue
8	4	2026-02-20	27000.00	Paid
9	5	2026-05-01	90000.00	Paid
10	5	2026-06-01	90000.00	Pending
11	6	2026-03-15	15000.00	Overdue
12	7	2026-06-01	84000.00	Paid
13	8	2026-04-20	33000.00	Paid
14	9	2026-07-01	60000.00	Paid
15	10	2026-05-15	30000.00	Paid
16	11	2026-08-01	150000.00	Paid
17	12	2026-06-10	18000.00	Paid
18	13	2026-09-01	54000.00	Paid
19	14	2026-07-15	30000.00	Overdue
20	15	2026-10-01	22000.00	Pending

(20 rows)

Все зависимости представлены ниже



Операторы языка SQL для вывода агрегатных данных в обновленной БД

1. Список клиентов, которые просрочили оплату. С указанием даты платежа и контактного человека (соединение таблиц)

```

SELECT contact_person, payment_date, payment_status FROM payments
JOIN rentals USING(rental_id) JOIN clients USING(client_id)
WHERE payment_status IN ('Overdue');

```

contact_person	payment_date	payment_status
Olga Ivanova	2026-05-01	Overdue
Elena Morozova	2026-03-15	Overdue
Dmitry Volkov	2026-07-15	Overdue

(3 rows)

2. Сколько суммарно поступало платежей в каждый месяц?
(скалярные функции работы с датой)

```
SELECT to_char(payment_date, 'MM.YYYY') AS month, SUM(amount) AS all_sum
FROM payments WHERE payment_status IN ('Paid') GROUP BY month;
```

month	all_sum
02.2026	72000.00
03.2026	24000.00
04.2026	177000.00
05.2026	120000.00
06.2026	102000.00
07.2026	60000.00
08.2026	150000.00
09.2026	54000.00

(8 rows)

3. Вывести клиентов из Москвы с номером телефона

```
SELECT client_id, contact_person, phone, address
FROM clients
WHERE address LIKE '%Moscow%';
```

client_id	contact_person	phone	address
1	Ivan Petrov	+79991112233	Moscow
12	Elena Volkova	+79995554433	Moscow

(2 rows)

4. Запрос на клиентов, у которых нет точек, переделано для новой БД (поиск родительских записей, у которых нет дочерних)

```
SELECT name, store_id AS store_id FROM clients
LEFT JOIN rentals USING(client_id) LEFT JOIN rental_stores USING(rental_id)
WHERE store_id IS NULL;
```

name	store_id
MyStore	
ElectroWorld	
Fashion	

(3 rows)

Триггер INSERT для проверки правильности вводимых данных

Условие: аренда не может быть короче 7 дней.

```
CREATE OR REPLACE FUNCTION check_duration()
RETURNS TRIGGER AS $$
BEGIN
    IF (NEW.end_date - NEW.start_date) < 7 THEN
        RAISE EXCEPTION 'Аренда должна быть не менее 7 дней';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_check_duration
BEFORE INSERT ON rentals
FOR EACH ROW
EXECUTE FUNCTION check_duration();
```

Проверка:

```
INSERT INTO rentals (client_id, start_date, end_date)
VALUES (1, '2025-06-01', '2025-06-05');
```

Ошибка:

Data Output	Messages	Notifications
ERROR: Аренда должна быть не менее 7 дней CONTEXT: PL/pgSQL function check_duration() line 4 at RAISE SQL state: P0001		

Процедура с курсором для модификации определенных записей по условию

Задача: повысить стоимость аренды для точек с кондиционером

```
CREATE OR REPLACE PROCEDURE increase_rent_for_has_ac(per NUMERIC)
LANGUAGE plpgsql AS $$
DECLARE
    cur CURSOR FOR
        SELECT store_id, daily_cost FROM stores
        WHERE has_ac = TRUE;
    store_rec RECORD;
BEGIN
    OPEN cur;
```

```

LOOP
  FETCH cur INTO store_rec;
  EXIT WHEN NOT FOUND;

  UPDATE stores
  SET daily_cost = daily_cost * (1 + per/100)
  WHERE store_id = store_rec.store_id;
END LOOP;
CLOSE cur;
END;
$$;

```

До вызова:

store_id	daily_cost
1	1500.00
3	2500.00
4	900.00
7	2800.00
9	2000.00
10	1000.00
11	5000.00

(7 rows)

Вызов: `CALL increase_rent_for_has_ac(10);`

Data Output	Messages	Notifications
CALL		
Query returned successfully in 77 msec.		

После вызова:

store_id	daily_cost
1	1650.00
3	2750.00
4	990.00
7	3080.00
9	2200.00
10	1100.00
11	5500.00

(7 rows)

Процедура выполнена верно.

Удаление родительской записи без CASCADE

```
CREATE OR REPLACE PROCEDURE delete_client_with_rentals(p_client_id INTEGER)
LANGUAGE plpgsql AS $$
BEGIN
    DELETE FROM payments WHERE rental_id IN (
        SELECT rental_id FROM rentals WHERE client_id = p_client_id
    );
    DELETE FROM rental_stores WHERE rental_id IN (
        SELECT rental_id FROM rentals WHERE client_id = p_client_id
    );
    DELETE FROM rentals WHERE client_id = p_client_id;
    DELETE FROM clients WHERE client_id = p_client_id;
END;
$$;
```

Проверим эту процедуру на примере клиента client_id = 8

Данные до удаления

Клиент: `SELECT * FROM clients WHERE client_id = 8;`

client_id	name	requisites	address	phone	contact_person
8	Ozon	333888555	Voronezh	+79263334455	Gromov Sergey

(1 row)

Его аренды: `SELECT * FROM rentals WHERE client_id = 8;`

rental_id	client_id	start_date	end_date
8	8	2026-03-20	2026-09-20

(1 row)

Платежи по этим арендам: `SELECT p.* FROM payments p
JOIN rentals r ON p.rental_id = r.rental_id WHERE r.client_id = 8;`

payment_id	rental_id	payment_date	amount	payment_status
13	8	2026-04-20	33000.00	Paid

(1 row)

Связи аренды с точками: `SELECT rs.* FROM rental_stores rs
JOIN rentals r ON rs.rental_id = r.rental_id WHERE r.client_id = 8;`

rental_store_id	rental_id	store_id
11	8	8

(1 row)

Выполним процедуру: `CALL delete_client_with_rentals(2);`

Data Output	Messages	Notifications
CALL		
Query returned successfully in 83 msec.		

Данные после удаления

Клиент: `SELECT * FROM clients WHERE client_id = 8;`

client_id	name	requisites	address	phone	contact_person
(0 rows)					

Его аренды: `SELECT * FROM rentals WHERE client_id = 8;`

rental_id	client_id	start_date	end_date
(0 rows)			

Платежи по этим арендам: `SELECT p.* FROM payments p
JOIN rentals r ON p.rental_id = r.rental_id WHERE r.client_id = 8;`

payment_id	rental_id	payment_date	amount	payment_status
(0 rows)				

Связи аренды с точками: `SELECT rs.* FROM rental_stores rs
JOIN rentals r ON rs.rental_id = r.rental_id WHERE r.client_id = 8;`

rental_store_id	rental_id	store_id
(0 rows)		

Процедура выполнялась верно.

Создание схемы EAV необязательных атрибутов

Создана таблица с типами атрибутов

```
CREATE TABLE attributes (  
  attr_id SERIAL PRIMARY KEY,  
  attr_name VARCHAR(100) UNIQUE NOT NULL  
);
```

Внесены 2 атрибута: персональная скидки и вебсайт компании клиента

```
insert into attributes values (1, 'discount_percent'), (2, 'website');  
select * from attributes;
```

attr_id	attr_name
1	discount_percent
2	website

Создана таблица со значением атрибутов для сущности - клиенты

```
CREATE TABLE entity_attr_values (  
  client_id INT REFERENCES clients(client_id),  
  attr_id INT REFERENCES attributes(attr_id),  
  attr_value TEXT,  
  PRIMARY KEY (entity_id, attr_id)  
);
```

Внесены некоторые значения атрибутов для нескольких клиентов

```
insert into attr_clients values (3, 1, 7), (7, 1, 10), (13, 1, 3),  
(1, 2, 'nike.com'), (2, 2, 'adidas.com'), (3, 2, 'zara.com'), (4, 2, 'mvideo.ru');  
select * from attr_clients;
```

client_id	attr_id	attr_value
3	1	7
7	1	10
13	1	3
1	2	nike.com
2	2	adidas.com
3	2	zara.com
4	2	mvideo.ru

(7 rows)

Демонстрация результатов через соединение таблицы клиентов с таблицей атрибутов

```
SELECT client_id, name, attr_id, attr_value FROM clients
```

```
LEFT JOIN attr_clients USING(client_id) ORDER BY client_id;
```

client_id	name	attr_id	attr_value
1	Nike	2	nike.com
2	Adidas	2	adidas.com
3	Zara	1	7
3	Zara	2	zara.com
4	MVideo	2	mvideo.ru
5	Puma		
6	Reebok		
7	H&M	1	10
9	Uniqlo		
10	Decathlon		
11	MyStore		
12	ElectroWorld		
13	Fashion	1	3

(13 rows)

Аналитический набор + рейтинг для торговых точек

Для сущности «Торговые точки» создан аналитический набор данных с помощью представления `store_rental_rating`. С использованием аналитической функции `RANK()` подсчитан рейтинг точек по ежедневной стоимости аренды, а с помощью функции `LEAD()` рассчитано опережение (разница в стоимости) между текущей точкой и следующей в рейтинге.

```
CREATE OR REPLACE VIEW store_rental_rating AS
```

```
SELECT
```

```
store_id, floor, area, daily_rental_cost,
```

```
RANK() OVER (ORDER BY daily_cost DESC) AS cost_rank,
```

```
daily_rental_cost-LEAD(daily_cost) OVER (ORDER BY daily_cost DESC) AS gap_to_next
```

```
FROM stores;
```

```
arenda=# select * from store_rental_rating;
```

store_id	floor	area	daily_cost	cost_rank	gap_to_next
11	1	200.00	5500.00	1	2420.00
7	2	95.00	3080.00	2	80.00
5	2	120.00	3000.00	3	250.00
3	1	78.30	2750.00	4	550.00
9	1	60.00	2200.00	5	550.00
1	1	45.50	1650.00	6	550.00
10	2	35.00	1100.00	7	0.00
8	3	42.00	1100.00	7	110.00
4	3	28.50	990.00	9	190.00
2	2	32.00	800.00	10	200.00
12	3	18.50	600.00	11	100.00
6	1	15.00	500.00	12	

(12 rows)